Vulnerability Scanning Techniques

Core Concepts

A. Scan Types

Vulnerability scanning identifies weaknesses in systems, networks, and applications using automated tools.

1. Network Scanning

Purpose: Discover hosts, open ports, and exposed services.

Common Tool: Nmap

Examples:

- Port scanning
- Service detection
- OS fingerprinting

Typical Commands

```
nmap -sS <target_IP>        # TCP SYN scan
nmap -sV <target_IP>        # Service version detection
nmap -O <target_IP>         # OS detection
```

It Finds:

- Open ports (22, 80, 443)
- Running services
- Misconfigured systems

2. Application (Web) Scanning

Purpose: Detect vulnerabilities in web applications.

Common Tool: Nikto

Example Command

nikto -h http://target_IP

Detects

- Outdated web servers
- Dangerous files
- SQL Injection indicators
- XSS vulnerabilities
- Weak configurations

3. Authenticated vs Unauthenticated Scans

Scan Type	Description	Accuracy
Unauthenticated Scan	External attacker perspective	Medium
Authenticated Scan	Uses login credentials	High

Authenticated scans detect:

- Missing patches
- Weak permissions
- Local vulnerabilities

Vulnerability Scoring (CVSS v4.0)

CVSS:

Common Vulnerability Scoring System (CVSS) measures severity of vulnerabilities.

Score Range

Score	Severity
0.0	None
0.1–3.9	Low
4.0–6.9	Medium
7.0–8.9	High
9.0–10.0	Critical

Example: Apache Struts RCE

Vulnerability: CVE-2017-5638
Affected Software: Apache Struts

Impact

- Remote Code Execution (RCE)
- Full server compromise

CVSS Score: ~10.0 (Critical)

Reason:

- Network exploitable
- No authentication required
- Complete system takeover possible

False Positives (Very Important Concept)

A scanner reports a vulnerability that **does not actually exist**.

It Happens

- Banner grabbing errors
- Version misidentification
- Patched systems showing vulnerable versions
- Network filtering

Validation Methods

Manual verification:

nmap -sV <target_IP>

Check service version manually:

curl -I http://target_IP

Verify exploitability:

- Try safe proof-of-concept checks
- Review vendor advisories

Key Objectives of Vulnerability Scanning

A cybersecurity analyst to learn

✔ Configure scan policies correctly
✔ Reduce false positives
✔ Interpret CVSS scores
✔ Prioritize remediation
✔ Validate vulnerabilities manually
✔ Produce accurate risk assessments

To Learn:

1. OWASP Testing Guide

Focus Areas:

- Web vulnerability testing methodology
- SQL Injection testing
- Authentication testing
- Session management flaws

Learn:

- Structured web scanning process
- Manual validation techniques

2. NIST SP 800-115

Official technical guide for security testing.

Covers:

- Planning scans
- Rules of engagement
- Vulnerability assessment lifecycle
- Reporting standards

Scanning is part of **security assessment**, not hacking.

3. Case Study — WannaCry Ransomware

Root Cause

- SMB vulnerability (MS17-010)
- Exploited EternalBlue exploit

Why Important for Learning

- Patch existed but systems were unscanned/unpatched.
- Demonstrates importance of vulnerability management.

CVSS Mapping

- Network exploitable
- No authentication
- High impact → Critical severity

Practical Learning Lab

Practice using:

Tool	Purpose
Nmap	Network discovery
OpenVAS / Greenbone	Full vulnerability scanner
Nikto	Web scanning
Metasploitable	Vulnerable target VM

Workflow:

1. Discover hosts → Nmap
2. Run vulnerability scan → OpenVAS
3. Validate results manually
4. Assign CVSS severity
5. Write report

Penetration Testing Techniques

1. **Core Concepts of Penetration Testing**

Penetration Testing (Pentest) is a **simulated cyberattack on a system, network, or application** to identify vulnerabilities before attackers exploit them.

Goal:

- Identify security weaknesses
- Exploit vulnerabilities safely
- Provide recommendations to improve security

Phases of Penetration Testing

1. **Reconnaissance (Information Gathering)**

Collect information about the target before attacking.

Types

Passive Recon

- No direct interaction with target
- Uses public information

Examples:

- Google search

- LinkedIn employee data
- DNS records

Active Recon

- Direct interaction with the target system

Examples:

- Ping sweep
- DNS enumeration
- Network scanning

Tools

- **Shodan** (search internet-connected devices)
- Maltego
- theHarvester
- WHOIS

Example:
Search exposed servers using Shodan.

2. **Scanning Phase**

   Identify open ports, services, and vulnerabilities.

Types of Scans

Network Scanning

- Discover devices and ports

Example:

```
nmap -sS <target-ip>
```

Vulnerability Scanning

Tools:

- Nessus
- OpenVAS
- Nikto

Finds:

- Misconfigurations
- Outdated software
- Known CVEs

3. **Exploitation Phase**

Use vulnerabilities to gain access to the system.

Popular Tool

Metasploit Framework

Example:

Msfconsole

Typical workflow:

1. Search exploit
2. Configure payload
3. Launch exploit

Example:

search vsftpd

use exploit/unix/ftp/vsftpd_234_backdoor

set RHOST <target-ip>

exploit

4. **Post-Exploitation**

Maintain access and gather deeper information.

Activities:

- Privilege escalation
- Password dumping
- Data exfiltration
- Network pivoting

Example commands:

getuid

sysinfo

hashdump

Privilege escalation tools:

- LinPEAS
- WinPEAS
- Mimikatz

5. **Reporting Phase**

Provide a **professional report to the client**.

Report includes:

1. Executive Summary
2. Vulnerability details
3. CVSS severity
4. Proof of concept
5. Remediation recommendations

Example entry:

Vulnerability	Severity	Recommendation
Outdated Apache Server	High	Update to latest version

1.Penetration Testing Methodologies

PTES provides a **structured pentesting framework**.

Phases:

1. Pre-engagement interactions
2. Intelligence gathering
3. Threat modeling
4. Vulnerability analysis
5. Exploitation
6. Post-exploitation
7. Reporting

Benefit:
Ensures **consistent and professional pentesting process**.

2. OWASP WSTG (Web Security Testing Guide)

Focus: **Web application security testing**

Tests include:

- SQL Injection
- Cross-Site Scripting (XSS)
- Authentication bypass
- Session management flaws
- Access control issues

Used heavily in **web pentests**.

Ethical Rules in Penetration Testing

Pentesting must always follow **ethical and legal guidelines**.

Rules:

✔ Get **written authorization**
✔ Follow **defined scope**
✔ Avoid damaging systems
✔ Maintain confidentiality
✔ Report findings responsibly

Unauthorized testing is **illegal hacking**.

Key Learning Objectives

A pentester should learn to:

✔ Perform structured penetration testing
✔ Use recon and scanning tools effectively
✔ Exploit vulnerabilities safely
✔ Perform privilege escalation
✔ Write professional security reports
✔ Follow ethical guidelines

1. Study PTES

Learn:

- Full pentesting lifecycle
- Real-world engagement structure

2.Study OWASP WSTG

Best resource for:

- Web penetration testing
- Practical vulnerability testing

3.Review SANS Case Studies

Learn from **real cyberattack investigations**:

Examples:

- Data breaches
- Ransomware incidents
- Insider threats

These help understand **real-world pentest scenarios**.

Lab Practice

Recon

```
whois target.com
```

Scan
```
nmap -A <target-ip>
```

Vulnerability Scan
Openvas

Exploit
Msfconsole

Privilege Escalation
linpeas.sh

Exploit Development

Exploit development is the process of creating code that **takes advantage of a vulnerability** in software to gain unauthorized access or perform unintended actions.

Purpose in cybersecurity:

- Understand how attacks work
- Help developers fix vulnerabilities
- Improve security testing

 Exploits should **only be tested in authorized lab environments** (Kali Linux, Metasploitable, TryHackMe labs).

Buffer Overflow

A **buffer overflow** occurs when a program writes more data into memory than the allocated buffer size.

Result:

- Overwrites adjacent memory
- Allows attackers to execute malicious code

Example

If a program expects **20 characters** but receives **100 characters**, the extra data may overwrite the program's memory.

Simple Concept

```
Buffer Size = 20 bytes
Input = 100 bytes
Extra data overwrites memory → program control
hijacked
```

Common targets:

- Old C programs
- Network services
- Embedded systems

SQL Injection

A vulnerability where attackers **inject malicious SQL queries** into application inputs.

Example vulnerable login query:

```
SELECT * FROM users WHERE username='admin' AND
password='123';
```

Attacker input:

admin' OR '1'='1

Resulting query:

SELECT * FROM users WHERE username='admin' OR '1'='1';

Effect:
✔ Authentication bypass
✔ Database access
✔ Data theft

Cross-Site Scripting (XSS)

XSS:

XSS occurs when **user input is not properly sanitized**, allowing attackers to inject malicious JavaScript.

Example vulnerable input field:

Comment: <script>alert("Hacked")</script>

Result:

- Script executes in victim's browser
- Session cookies may be stolen
- Website defacement

Types:

- Stored XSS
- Reflected XSS
- DOM-based XSS

Exploit Writing Basics

Exploit developers often write scripts using **Python, C, or Ruby**.

Basic Exploit Workflow

1 Identify vulnerability
2 Analyze program behavior
3 Craft exploit payload
4 Execute payload
5 Gain system control

Example: Simple Python Buffer Overflow Payload

buffer = "A" * 1000

print(buffer) Purpose:

- Send large input to crash program
- Determine overflow length

Later stages include:

- Finding **offset**
- Injecting **shellcode**
- Redirecting execution

Using Exploit-DB

Exploit Database (Exploit-DB) provides **public proof-of-concept exploits**.

Typical workflow:

1. Search exploit
2.
3. Download PoC
4. Modify exploit
5. Test in lab

Example search in Kali:

searchsploit apache

Example output:

Apache Struts RCE exploit

Apache Tomcat exploit

Copy exploit:

searchsploit -m exploit_name

Exploit Mitigation Techniques

Modern systems include protections to prevent exploits.

1. ASLR (Address Space Layout Randomization)

Randomizes memory addresses.

Result:

- Makes buffer overflow attacks harder
- Exploit cannot predict memory location

2. WAF (Web Application Firewall)

Protects web applications by filtering malicious traffic.

Detects:

- SQL injection
- XSS
- Malicious payloads

Example WAFs:

- Cloudflare WAF
- ModSecurity

3. Patching

Most effective defense.

Developers fix vulnerabilities through **software updates**.

Example:

- Patch removes vulnerable code
- Prevents exploit execution

Key Learning Objectives

A cybersecurity learner should be able to:

✔ Understand common exploit types
✔ Identify vulnerable code
✔ Write basic exploit scripts
✔ Use proof-of-concept exploits safely
✔ Understand mitigation mechanisms

How to Learn Exploit Development

1 Study Exploit-DB

Learn from **real exploit examples**.

Practice:

searchsploit vsftpd

Analyze exploit code to understand:

- Payload structure
- Exploit logic

Follow TCM Security Guides

TCM courses teach:

- Binary exploitation
- Buffer overflow attacks
- Real pentesting scenarios

Good for beginners.

Practice in TryHackMe

Recommended room:

Buffer Overflow Prep

Skills learned:

- Finding offsets
- Exploit development
- Shellcode injection

Safe lab environment.

Lab Setup

For safe testing:

Tool	Purpose
Kali Linux	Attacker machine
Metasploitable	Vulnerable target
TryHackMe	Guided labs
Exploit-DB	PoC exploits

VULNERABILITY SCANNING LAB

Tools: Nmap, OpenVAS, Nikto

- **Nmap**

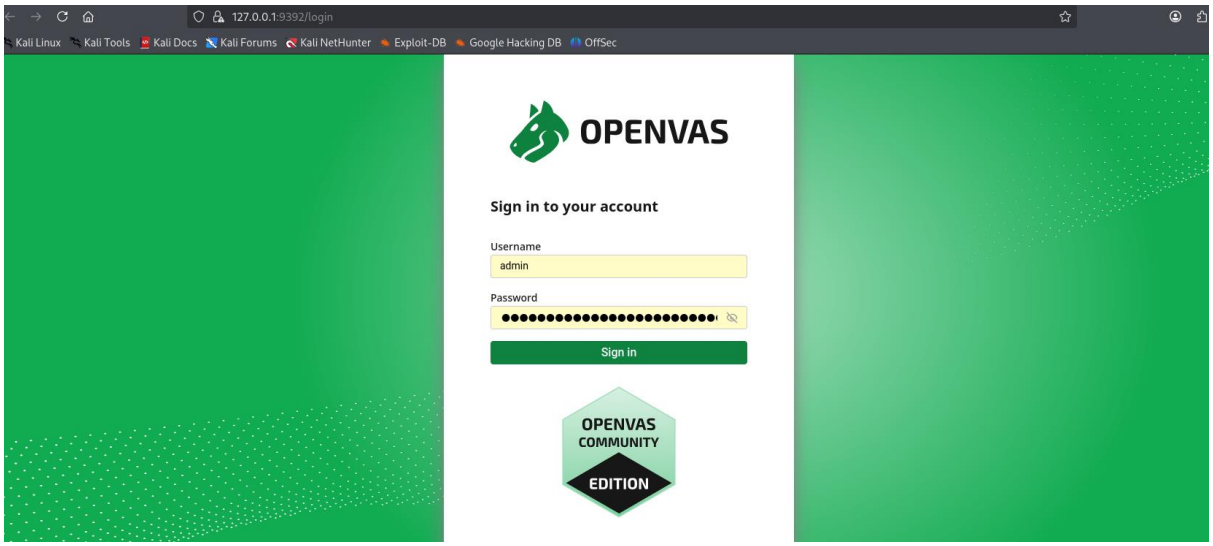
```
┌──(root㉿kali)-[/home/kali]
└─# nmap -sV 192.168.159.130
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-08 16:13 +0530
Nmap scan report for 192.168.159.130
Host is up (0.0098s latency).
Not shown: 977 filtered tcp ports (no-response)
PORT     STATE SERVICE      VERSION
21/tcp   open  ftp          vsftpd 2.3.4
22/tcp   open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet       Linux telnetd
25/tcp   open  smtp         Postfix smtpd
53/tcp   open  domain       ISC BIND 9.4.2
80/tcp   open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind      2 (RPC #100000)
139/tcp  open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec?
513/tcp  open  login?
514/tcp  open  shell?
1099/tcp open  java-rmi     GNU Classpath grmiregistry
1524/tcp open  bindshell    Metasploitable root shell
2049/tcp open  nfs          2-4 (RPC #100003)
2121/tcp open  ccproxy-ftp?
3306/tcp open  mysql?
5432/tcp open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc          VNC (protocol 3.3)
6000/tcp open  X11          (access denied)
6667/tcp open  irc          UnrealIRCd
8009/tcp open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp open  unknown
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 177.68 seconds
```

- **Nikto**

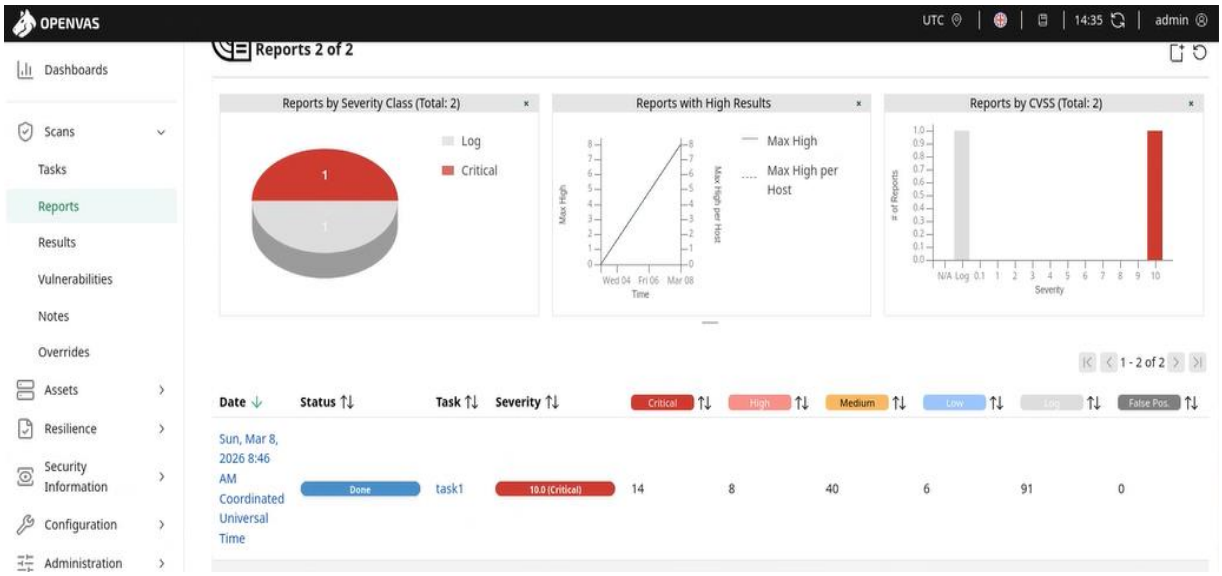
```
┌──(root㉿kali)-[/home/kali]
└─# nikto -h 192.168.159.130
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          192.168.159.130
+ Target Hostname:    192.168.159.130
+ Target Port:        80
+ Start Time:         2026-03-08 16:17:02 (GMT5.5)
---------------------------------------------------------------------------
+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ /: Server may leak inodes via ETags, header found with file /, inode: 67670, size: 10703, mtime: Sun May 25 17:46:56 2025.
See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Head
ers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a dif
ferent fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-ty
pe-header/
+ /index.php?: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The fol
lowing alternatives for 'index' were found: index.html, index.php. See: http://www.wisec.it/sectou.php?id=4698ebdc59d15,https
://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ Multiple index files found: /index.html, /index.php.
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, OPTIONS, TRACE .
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/C
ross_Site_Tracing
+ /phpinfo.php: Output from the phpinfo() function was found.
+ /doc/: Directory indexing found.
+ /doc/: The /doc/ directory is browsable. This may be /usr/doc. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-
0678
+ /phpMyAdmin/changelog.php: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hos
ts.
+ /phpMyAdmin/ChangeLog: phpMyAdmin is for managing MySQL databases, and should be protected or limited to authorized hosts.
+ /test/: Directory indexing found.
+ /test/: This might be interesting.
+ /phpinfo.php: PHP is installed, and a test script which runs phpinfo() was found. This gives a lot of system information. S
ee: CWE-552
```

- **OpenVAS**

```
┌──(root㉿kali)-[/home/kali]
└─# gvm-start
[>] Please wait for the GVM services to start.
[>]
[>] You might need to refresh your browser once it opens.
[>]
[>]  Web UI (Greenbone Security Assistant): https://127.0.0.1:9392

● gsad.service - Greenbone Security Assistant daemon (gsad)
     Loaded: loaded (/usr/lib/systemd/system/gsad.service; disabled; preset: disabled)
     Active: active (running) since Sun 2026-03-08 16:43:13 IST; 28ms ago
 Invocation: 9667f9a1c74d44d19d80ae8e178024f4
       Docs: man:gsad(8)
             https://www.greenbone.net
   Main PID: 7919 (gsad)
      Tasks: 1 (limit: 18907)
     Memory: 2.2M (peak: 2.2M)
        CPU: 45ms
     CGroup: /system.slice/gsad.service
             └─7919 /usr/sbin/gsad --foreground --listen 127.0.0.1 --port 9392

Mar 08 16:43:13 kali systemd[1]: Starting gsad.service - Greenbone Security Assistant daemon (gsad)...
Mar 08 16:43:13 kali systemd[1]: Started gsad.service - Greenbone Security Assistant daemon (gsad).
Mar 08 16:43:13 kali gsad[7919]: gsad main:MESSAGE:2026-03-08 11h13.13 utc:7919: Starting GSAD version 24.12.2~git

● gvmd.service - Greenbone Vulnerability Manager daemon (gvmd)
     Loaded: loaded (/usr/lib/systemd/system/gvmd.service; disabled; preset: disabled)
     Active: active (running) since Sun 2026-03-08 16:43:08 IST; 5s ago
 Invocation: 8393737f78ea4d55a97035d5e65d0146
       Docs: man:gvmd(8)
    Process: 7829 ExecStart=/usr/sbin/gvmd --osp-vt-update=/run/ospd/ospd-openvas.sock --listen-group=_gvm (code=exited, stat
us=0/SUCCESS)
   Main PID: 7831 (gvmd)
      Tasks: 3 (limit: 18907)
     Memory: 420.9M (peak: 501.6M)
        CPU: 14.597s
     CGroup: /system.slice/gvmd.service
             ├─7831 "gvmd: Waiting " --osp-vt-update=/run/ospd/ospd-openvas.sock --listen-group=_gvm
```



Run scans, prioritize vulnerabilities, document results



OpenVAS Scan Findings

Example vulnerabilities detected:

Vulnerability	CVE ID	CVSS
Apache Path Traversal	CVE-2021-41773	9.8
Samba Remote Code Execution	CVE-2007-2447	10.0
FTP Anonymous Login	N/A	7.5

Risk Prioritization

Priority	CVSS Range
Critical	9.0 – 10
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9

Example:

- **SQL Injection (9.1)** → Critical
- **Apache Path Traversal (9.8)** → Critical
- **SMB Port Exposure (6.5)** → Medium

Report

Title

Critical Web Vulnerabilities in Metasploitable2 Server

Executive Summary

A vulnerability assessment was conducted on the Metasploitable2 server using Nmap, OpenVAS, and Nikto. Multiple critical vulnerabilities were discovered including SQL Injection, outdated Apache server, and exposed SMB services. These vulnerabilities could allow attackers to gain unauthorized access or execute remote code.

Technical Findings

Vulnerability: Apache Path Traversal
CVE: CVE-2021-41773
Host: 192.168.159.130
CVSS Score: 9.8 (Critical)

Description:
The Apache web server version is vulnerable to a path traversal attack allowing attackers to access restricted files.

Remediation

- Update Apache to the latest version
- Disable unused services (Telnet, FTP)
- Restrict SMB access
- Apply security patches
- Implement firewall rules

Escalation Email

Subject Critical Vulnerabilities Detected on Metasploitable Server

Dear Development Team,

During a recent vulnerability scan of the server (192.168.1.100), several critical security issues were identified. The most severe vulnerability is Apache Path Traversal (CVE-2021-41773) with a CVSS score of 9.8. This flaw may allow attackers to access sensitive files and potentially execute malicious commands.

Additionally, exposed services such as Telnet and FTP with anonymous login increase the risk of unauthorized access. Proof of concept testing confirmed that these services are accessible remotely.

We recommend immediately patching the Apache server, disabling unused services, and applying security updates to mitigate these risks.

Regards,
Security Team

RECONNAISSANCE

Domain Information

- **Target Domain:** example.com
- **IP Address:** 192.168.159.128
- **Hosting Provider:** Local Lab / Internal Network
- **WHOIS Information:** Domain registration details collected using WHOIS lookup.

Subdomains Identified

Subdomain	Method	Status
dev.example.com	Sublist3r	Active
test.example.com	Sublist3r	Active
mail.example.com	DNS Enumeration	Active

Exposed Services

IP Address	Port	Service	Tool
192.168.159.128	22	SSH	Shodan
192.168.159.128	80	HTTP	Nmap
192.168.159.128	443	HTTPS	Nmap

Asset Mapping Log

Timestamp	Tool	Finding
2026-03-08 10:00:00	Shodan	Exposed SSH service on 192.168.159.128
2026-03-08 10:15:00	WHOIS	Retrieved domain registration information
2026-03-08 10:30:00	Maltego	Discovered subdomain dev.example.com
2026-03-08 10:40:00	Sublist3r	Enumerated additional subdomains
2026-03-08 10:50:00	Wappalyzer	Identified Apache and PHP technology stack

Summary

- Reconnaissance was performed using OSINT tools including Shodan, Maltego, and Sublist3r. The target domain and associated IP were identified, along with multiple active subdomains.

Several exposed services such as SSH and HTTP were discovered. Technology stack analysis revealed the web server and framework used by the target system.

EXPLOITATION LAB

Exploit Simulation

Target: Metasploitable2
Attacker Machine: Kali Linux
Tool Used: Metasploit Framework

Module Used

exploit/multi/http/tomcat_mgr_login

Target IP

192.168.159.130

Payload

java/jsp_shell_reverse_tcp

Exploit Results Log

Exploit ID	Description	Target IP	Status	Payload
003	Apache Tomcat Manager RCE	192.168.159.130	Success	Java Reverse Shell
004	SQL Injection Test	192.168.159.130	Success	Database Dump
005	Command Injection Test	192.168.159.130	Failed	N/A

SQL Injection Test (sqlmap)

Example command:

sqlmap -u "http://192.168.159.130/login.php?id=1" --dbs

Burp Suite Validation

Steps:

1. Intercept login request.
2. Send request to **Repeater**.
3. Modify parameter values.

4. Observe server response.

Finding

Parameter `id` vulnerable to **SQL Injection**.

Exploit-DB Validation

Exploit verified using public PoC from **Exploit-DB**.

Example search:

searchsploit tomcat manager

Summary

- Exploitation testing was performed on the Metasploitable2 target using Metasploit, sqlmap, and Burp Suite. A successful remote code execution exploit was achieved through the Apache Tomcat Manager interface. SQL injection vulnerabilities were also validated. Exploit-DB proof-of-concept references confirmed the vulnerabilities, demonstrating practical exploitation and verification techniques.

POST-EXPLOITATION PRACTICE

Privilege Escalation

Tool Used: Metasploit Meterpreter

Module Used:

exploit/windows/local/bypassuac

System Information Collection

Commands used:

sysinfo
getuid
ipconfig
netstat -ano

Evidence Collection

Evidence files were collected from the compromised system and hashed to maintain **integrity verification**.

Hash Command

sha256sum target.conf

Evidence Log

Item	Description	Collected By	Date	Hash Value
Config File	target.conf	VAPT Analyst	2025-08-18	5e884898da28047151d0e56f8dc6292773603d0d6aabbddc7f5c5b1e6a9a6b21
Log File	system.log	VAPT Analyst	2025-08-18	9c56cc51b374c3ba189210d5b6d4bf57790c2c4fdf4e9a8e2f8f1c1c8e27c3f5

Memory Analysis (Volatility)

Example command:

volatility -f memorydump.raw imageinfo

Used for:

- Detecting malicious processes
- Investigating memory artifacts
- Identifying persistence mechanisms

Post-Exploitation Summary

Post-exploitation activities were conducted using Meterpreter after successful system compromise. Privilege escalation was achieved using a UAC bypass exploit, granting SYSTEM-level access. System information and network details were collected. Evidence files were extracted and verified using SHA-256 hashing. Memory analysis using Volatility helped identify potential artifacts for forensic investigation.

CAPSTONE PROJECT – FULL VAPT CYCLE

Simulation – SQL Injection Exploit (DVWA)

Target Application: DVWA (Damn Vulnerable Web Application)
Target IP: 192.168.159.128
Tool Used: sqlmap (Kali Linux)

Command Used

sqlmap -u "http://192.168.159.128/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSES

Detection – OpenVAS Findings

Timestamp	Target IP	Vulnerability	PTES Phase
2025-08-18 12:00:00	192.168.159.128	Cross-Site Scripting (XSS)	Exploitation
2025-08-18 12:10:00	192.168.159.128	SQL Injection	Exploitation
2025-08-18 12:20:00	192.168.159.128	Weak HTTP Headers	Vulnerability Analysis

Remediation Recommendations

Recommended security fixes include:

- Implement **input validation and sanitization** for all user inputs.
- Use **prepared statements or parameterized queries** to prevent SQL injection.
- Apply **output encoding** to mitigate Cross-Site Scripting (XSS).
- Enable **secure HTTP headers** such as Content-Security-Policy and X-Frame-Options.
- Update and patch vulnerable software.
- Perform a **follow-up vulnerability scan using OpenVAS** after remediation.

PTES Report

A full vulnerability assessment and penetration testing (VAPT) cycle was conducted against a DVWA target environment using Kali Linux tools. The reconnaissance and scanning phases identified exposed services and potential web application vulnerabilities. OpenVAS scanning detected multiple issues including Cross-Site Scripting and insecure HTTP configurations.

During the exploitation phase, sqlmap was used to test the DVWA SQL injection module. The attack successfully extracted database information, confirming that the application was vulnerable to SQL injection. Enumeration revealed several databases including the DVWA application database, demonstrating the potential impact of the vulnerability.

Further testing confirmed the presence of Cross-Site Scripting vulnerabilities due to improper input validation. These weaknesses could allow attackers to execute malicious scripts in user browsers, potentially leading to session hijacking or data theft.

Remediation recommendations include implementing proper input validation, using parameterized database queries, enabling secure HTTP headers, and applying regular security updates. A rescan using OpenVAS is recommended after remediation to confirm that vulnerabilities have been properly addressed.

This exercise demonstrates the full PTES cycle from reconnaissance to reporting and highlights the importance of secure coding practices and continuous vulnerability management.

Non-Technical Briefing

A security assessment was conducted on a test web application to evaluate its resistance to cyberattacks. The testing identified several weaknesses, including vulnerabilities that allowed unauthorized access to database information and the possibility of malicious scripts being executed in the browser. These issues occur due to insufficient validation of user input. If exploited by attackers, they could lead to data exposure or compromise of user accounts. Recommended fixes include improving input validation, strengthening application security controls, and applying regular security updates. After implementing these improvements, a follow-up security scan should be performed to ensure that the vulnerabilities have been successfully resolved.